Date:10-10-2026

CO2-ASSESSMENT TOOL 1-ANALYTICAL PROBLEM SOLVING

COURSE CODE: DSA0308

COURSE NAME: NATURAL LANGUAGE PROCESSING

REGISTRATION NO:192472193 **NAME:** C G CHARISHMA

Question 1: Morphological Analysis Pipeline for a Document Indexing System

1. Aim

To develop a Python-based morphological analysis pipeline that decomposes connected, connecting and connection into their root and suffix components, classifies the suffixes as inflectional or derivational, and normalizes the words for document indexing.

2. Theory

Morphological analysis identifies the internal structure of words.

- **Root:** The core word carrying the main meaning.
- **Suffix:** A word element added to the end of a root.
- **Inflectional morphology:** Changes grammatical form without creating a new lexical word, such as connect + ed.
- **Derivational morphology:** Forms a related lexical word, such as connect + ion.
- **Normalization:** Maps related words to a common representation to improve information retrieval.

For this dataset, the common root is connect.

3. Algorithm

- Store the three input words.
- Convert each word to lowercase for consistent processing.
- Match each word against the predefined morphological rules.
- Identify the root and suffix.

- Classify -ed and -ing as inflectional in these examples and -ion as derivational.
- Record the part-of-speech information where applicable.
- Normalize all three words to connect.
- Display the decomposition, classification, intermediate steps and normalized form in a table.

4. Python Code

```
def analyze_word(word):
    rules = {
        "connected": {
            "root": "connect",
            "suffix": "-ed",
            "type": "Inflectional",
            "pos": "Verb (VBN)"
        },
        "connecting": {
            "root": "connect",
            "suffix": "-ing",
            "type": "Inflectional",
            "pos": "Verb (VBG)"
        },
        "connection": {
            "root": "connect",
            "suffix": "-ion",
            "type": "Derivational",
            "pos": "Noun (NN)"
        }
```

```
    }

    word = word.lower()
    if word not in rules:
        return None

    data = rules[word]

    print("\nWord:", word)
    print("Step 1: Identify the word.")
    print("Step 2: Match the morphological rule.")
    print("Step 3: Root =", data["root"])
    print("Step 4: Suffix =", data["suffix"])
    print("Step 5: Classification =", data["type"])
    print("Step 6: POS tag =", data["pos"])
    print("Step 7: Normalized form =", data["root"])

    return [
        word, data["root"], data["suffix"],
        data["type"], data["pos"], data["root"]
    ]


words = ["connected", "connecting", "connection"]

print("MORPHOLOGICAL ANALYSIS FOR DOCUMENT INDEXING")
```

```
print("=" * 100)

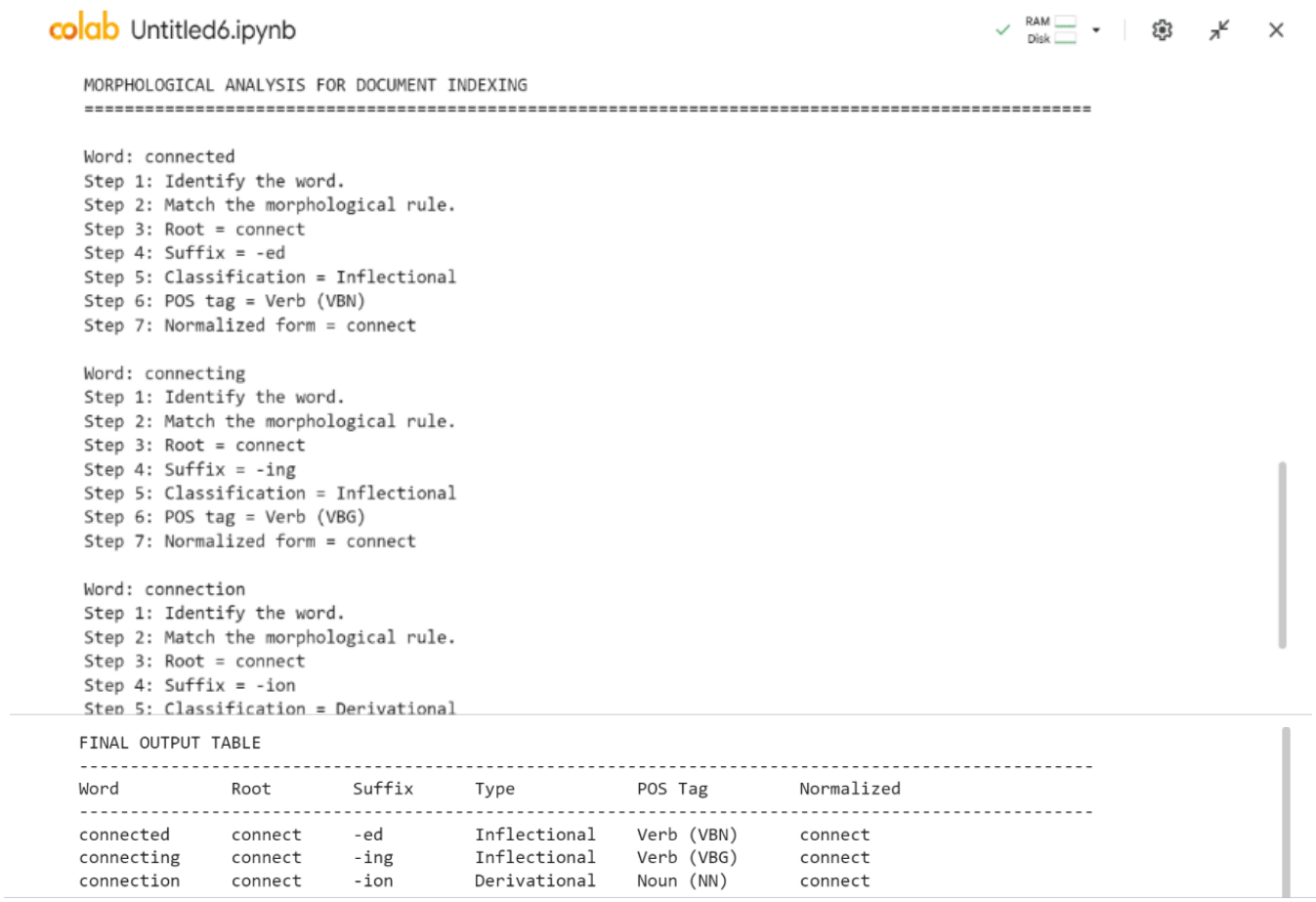
results = []

for word in words:
    result = analyze_word(word)
    if result:
        results.append(result)

print("\nFINAL OUTPUT TABLE")
print("-" * 100)
print(f"{'Word':<15}{'Root':<12}{'Suffix':<12}"
      f"{'Type':<16}{'POS Tag':<16}{'Normalized':<15}")
print("-" * 100)

for row in results:
    print(f"{row[0]:<15}{row[1]:<12}{row[2]:<12}"
          f"{row[3]:<16}{row[4]:<16}{row[5]:<15}")
```

5.Output



6. Result

The morphological analysis program successfully decomposes the three input words, classifies their suffixes and maps them to the common root connect.

7. Justification

- connected: connect + -ed; the suffix represents a grammatical verb form, so it is classified as inflectional.
- connecting: connect + -ing; the suffix represents a verb form, so it is classified as inflectional.
- connection: connect + -ion; the suffix forms a related noun, so it is classified as derivational.
- All three are normalized to connect, allowing documents containing these forms to be retrieved using a shared index term.

The tags VBN, VBG and NN are conventional Penn Treebank-style tags for past participle, gerund/present participle and noun, respectively. The program uses predefined rules for these specific words; it is not a general-purpose morphological analyzer.

Question 2: Morphological Parsing for Sentiment Analysis

1. Aim

To implement a rule-based morphological parsing module for unhappy, happiness and happily, identifying prefixes, suffixes, base forms, transformation types and normalized roots for sentiment analysis.

2. Theory

Derivational morphology creates related words by adding prefixes or suffixes.

- un- expresses negation.
- -ness converts an adjective into a noun.
- -ly commonly forms an adverb from an adjective.
- The spelling of happy changes from y to i before -ness and -ly.
- The shared lexical base is happy. However, normalization must not erase the semantic effect of the prefix un-.

3. Algorithm

- Accept the three words.
- Match each word to a rule-based morphological pattern.
- Identify its prefix, base, and suffix.
- Show any spelling change between the base and the surface form.
- Classify the transformation as derivational or inflectional.
- Assign an appropriate part-of-speech tag.
- Normalize the lexical root to happy.
- Preserve the negation information for unhappy.
- Display the analysis and final table.

4. Python Code

```
def analyze_word(word):
    rules = {
        "unhappy": {
            "prefix": "un-",
            "base": "happy",
            "suffix": "None",
            "change": "un- + happy",
            "type": "Derivational",
            "pos": "Adjective (JJ)",
            "meaning": "Negative meaning"
        },
        "happiness": {
            "prefix": "None",
            "base": "happy",
            "suffix": "-ness",
            "change": "happy -> happi + ness",
            "type": "Derivational",
            "pos": "Noun (NN)",
            "meaning": "State of being happy"
        },
        "happily": {
            "prefix": "None",
            "base": "happy",
            "suffix": "-ly",
            "change": "happy -> happi + ly",
```

```
        "type": "Derivational",
        "pos": "Adverb (RB)",
        "meaning": "In a happy manner"
    }
}

word = word.lower()
if word not in rules:
    return None

data = rules[word]

print("\nWord:", word)
print("Step 1: Identify prefix =", data["prefix"])
print("Step 2: Identify base =", data["base"])
print("Step 3: Identify suffix =", data["suffix"])
print("Step 4: Morphological form =", data["change"])
print("Step 5: Transformation =", data["type"])
print("Step 6: POS tag =", data["pos"])
print("Step 7: Semantic interpretation =", data["meaning"])
print("Step 8: Normalized root =", data["base"])

return [
    word, data["prefix"], data["base"], data["suffix"],
    data["type"], data["pos"], data["meaning"], data["base"]
]
```

```
words = ["unhappy", "happiness", "happily"]

print("MORPHOLOGICAL PARSING FOR SENTIMENT ANALYSIS")
print("=" * 125)

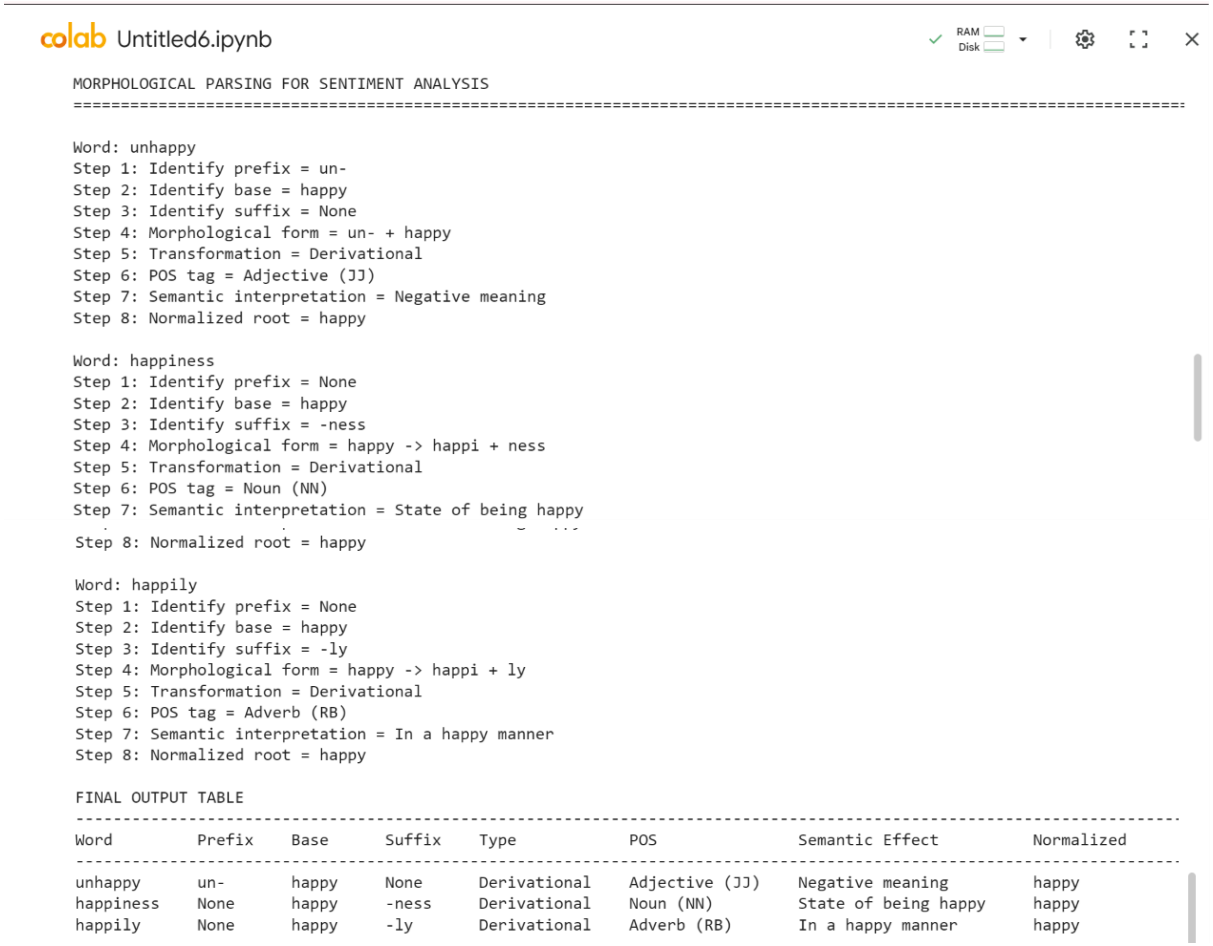
results = []

for word in words:
    result = analyze_word(word)
    if result:
        results.append(result)

print("\nFINAL OUTPUT TABLE")
print("-" * 125)
print(f"{'Word':<13}{'Prefix':<10}{'Base':<10}{'Suffix':<10}"
      f"{'Type':<16}{'POS':<18}{'Semantic Effect':<25}{'Normalized':<12}")
print("-" * 125)

for row in results:
    print(f"{row[0]:<13}{row[1]:<10}{row[2]:<10}{row[3]:<10}"
          f"{row[4]:<16}{row[5]:<18}{row[6]:<25}{row[7]:<12}")
```

5.Output



colab Untitled6.ipynb

```
MORPHOLOGICAL PARSING FOR SENTIMENT ANALYSIS
================================================================================================================================

Word: unhappy
Step 1: Identify prefix = un-
Step 2: Identify base = happy
Step 3: Identify suffix = None
Step 4: Morphological form = un- + happy
Step 5: Transformation = Derivational
Step 6: POS tag = Adjective (JJ)
Step 7: Semantic interpretation = Negative meaning
Step 8: Normalized root = happy

Word: happiness
Step 1: Identify prefix = None
Step 2: Identify base = happy
Step 3: Identify suffix = -ness
Step 4: Morphological form = happy -> happi + ness
Step 5: Transformation = Derivational
Step 6: POS tag = Noun (NN)
Step 7: Semantic interpretation = State of being happy
Step 8: Normalized root = happy

Word: happily
Step 1: Identify prefix = None
Step 2: Identify base = happy
Step 3: Identify suffix = -ly
Step 4: Morphological form = happy -> happi + ly
Step 5: Transformation = Derivational
Step 6: POS tag = Adverb (RB)
Step 7: Semantic interpretation = In a happy manner
Step 8: Normalized root = happy

FINAL OUTPUT TABLE
--------------------------------------------------------------------------------------------------------------------------------
Word         Prefix    Base      Suffix    Type            POS              Semantic Effect          Normalized
--------------------------------------------------------------------------------------------------------------------------------
unhappy      un-       happy     None      Derivational    Adjective (JJ)   Negative meaning         happy
happiness    None      happy     -ness     Derivational    Noun (NN)        State of being happy     happy
happily      None      happy     -ly       Derivational    Adverb (RB)      In a happy manner        happy
```

6. Result

The program identifies the morphological components of all three words and normalizes them to the lexical root happy.

7. Justification

- unhappy: The prefix un- reverses or negates the positive meaning of happy. A sentiment system should preserve this negation even though the lexical root is normalized.
- happiness: The suffix -ness forms a noun. The spelling changes from y to i, producing happiness.
- happily: The suffix -ly forms an adverb, with the same y-to-i spelling adjustment.

All transformations are derivational. Normalizing the words to happy supports lexical grouping, but sentiment analysis should retain the original form or negation feature to avoid treating unhappy as positive.

Question 3: Rule-Based Stemming for Search Engine Preprocessing

1. Aim

To develop a rule-based stemming module for played, player and playing, strip appropriate suffixes, distinguish inflectional and derivational forms, and generate a common normalized form for search indexing.

2. Theory

Stemming removes affixes to reduce related word forms to a common stem.

- played contains the inflectional suffix -ed.
- playing contains the inflectional suffix -ing.
- player contains the derivational suffix -er, which forms an agent noun.
- All three words can be mapped to play for this dataset.

A stemmer is not necessarily a dictionary lemmatizer. It applies rules to word forms and may produce stems that are not always complete dictionary words.

3. Algorithm

- Store the input words.
- Convert each word to lowercase.
- Check the word against the ordered suffix rules.
- Test -ing, -ed and -er as applicable.
- Remove the matching suffix.
- Record the intermediate stem and removed affix.
- Classify -ed and -ing as inflectional and -er as derivational in these examples.
- Normalize each word to play.
- Display the original word, suffix removed, intermediate stem, transformation type and final normalized form.

4. Python Code

```
def stem_word(word):
    word = word.lower()

    print("\nProcessing:", word)
    print("Step 1: Convert the word to lowercase ->", word)

    rules = [
        ("ing", "Inflectional"),
        ("ed", "Inflectional"),
        ("er", "Derivational")
    ]

    for suffix, category in rules:
        print("Step 2: Check suffix -" + suffix)

        if word.endswith(suffix):
            stem = word[:-len(suffix)]

            print("Step 3: Suffix found -> -" + suffix)
            print("Step 4: Remove suffix:", word, "->", stem)
            print("Step 5: Transformation type =", category)
            print("Step 6: Normalized form =", stem)

            return [
                word, stem, "-" + suffix,
```

```
            category, stem
        ]

    print("No matching suffix found.")
    return [word, word, "None", "No change", word]


words = ["played", "player", "playing"]

print("RULE-BASED STEMMING FOR SEARCH ENGINE PREPROCESSING")
print("=" * 95)

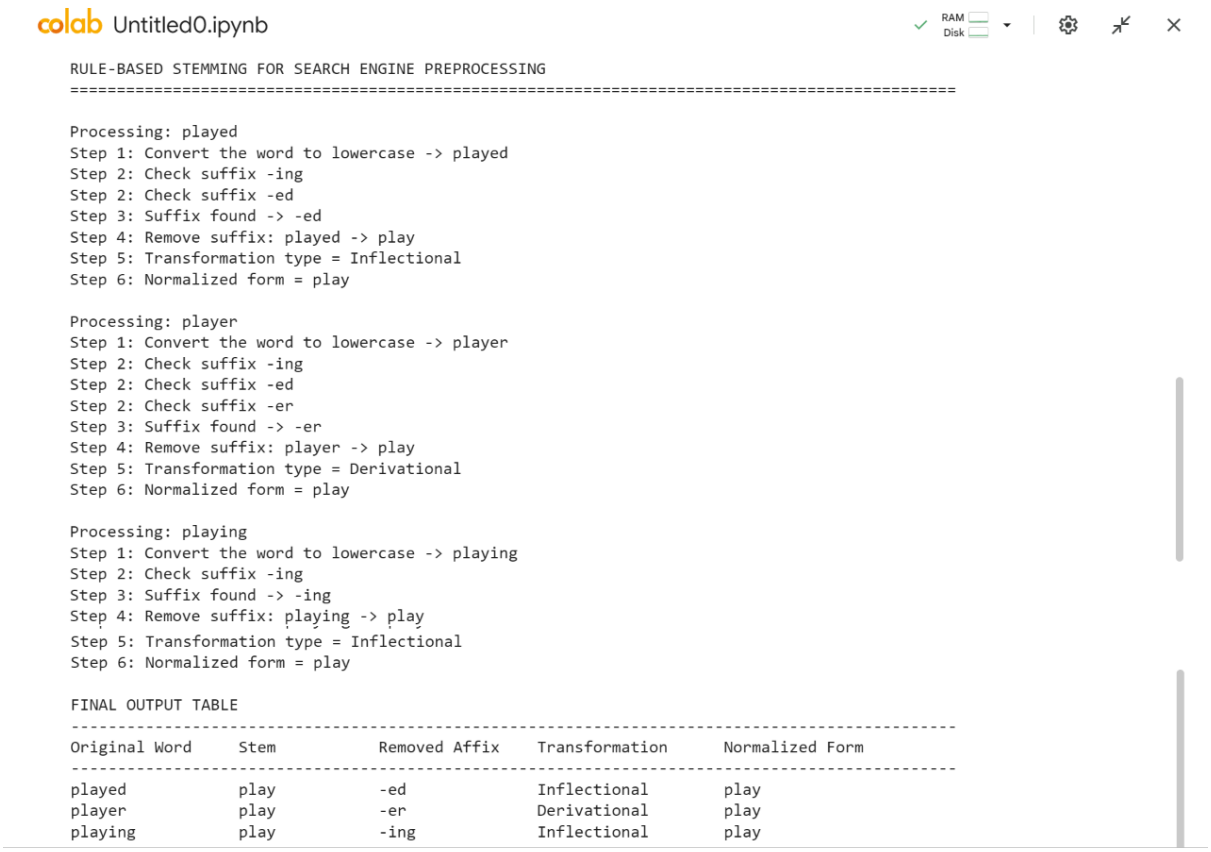
results = []

for word in words:
    results.append(stem_word(word))

print("\nFINAL OUTPUT TABLE")
print("-" * 95)
print(f"{'Original Word':<18}{'Stem':<15}{'Removed Affix':<17}"
      f"{'Transformation':<20}{'Normalized Form':<18}")
print("-" * 95)

for row in results:
    print(f"{row[0]:<18}{row[1]:<15}{row[2]:<17}"
          f"{row[3]:<20}{row[4]:<18}")
```

5.Output



6. Result

The stemming module successfully extracts play from played, player and playing, identifies the removed suffixes and classifies the transformations.

Justification

Word	composition	Classification	Justification
played	play + -ed	Inflectional	The suffix marks a past-tense verb form.
player	play + -er	Derivational	The suffix forms a noun referring to someone who plays.
playing	play + -ing	Inflectional	The suffix marks a grammatical verb form.

All three words are normalized to play. This allows a search engine to group documents containing these forms under a common index term.

Limitation: This is a small rule-based stemmer designed for the given words. A general-purpose stemmer would require additional rules and exception handling.

Question 4: Finite-State Morphological Parsing

1. Aim

To implement a finite-state morphological parser for writes, writing and written, model regular and irregular verb forms using state transitions, and normalize the forms to the common root write.

2. Theory

A finite-state morphological parser represents word analysis as transitions between states.

- **Initial state (q0):** Begins processing the input.
- **Root-analysis state (q1):** Checks whether the input matches a known verb pattern.
- **Regular-form states:** Handle regular forms such as writes and writing.
- **Irregular-form state:** Handles an irregular form such as written.
- **Final state (qf):** Accepts a recognized form and returns its analysis.

The parser must recognize that written is an irregular past participle rather than treating it as a regular write + -en spelling operation.

3. Algorithm/Procedure

- Define the input words and their known morphological analyses.
- Initialize the parser in state q0.
- Move to the root-analysis state q1.
- Check the input against the regular and irregular patterns.
- For writes, follow the regular third-person singular path.

- For writing, follow the regular present-participle path, accounting for the spelling change that removes the final e before -ing.
- For written, follow the irregular-form path.
- Move each recognized word to the final state qf.
- Record the transition path, morphological components, root, classification, POS tag and normalized form.
- Display the transition trace and results.

Finite-state transition model

Current state	Condition	Next state	Decision
q0	Input received	q1	Begin analysis
q1	Word is writes	_regular_s	Regular verb form
q1	Word is writing	regular_ing	Regular verb form with spelling change
q1	Word is written	q_irregular	Irregular verb form
q_regular_s	Analysis complete	qf	Accept
q_ regular_ing	Analysis complete	qf	Accept
q_irregular	Analysis complete	qf	Accept
q1	No known pattern	q_reject	Reject as unrecognized

4. Python Code

```
def parse_word(word):
    word = word.lower()

    print("\nProcessing:", word)
    path = ["q0"]
    print("Step 1: Start state = q0")

    path.append("q1")
    print("Step 2: Move to q1 for morphological analysis")

    patterns = {
        "writes": {
            "state": "q_regular_s",
            "components": "write + -s",
            "root": "write",
            "type": "Regular",
            "pos": "Verb (VBZ)"
        },
        "writing": {
            "state": "q_regular_ing",
            "components": "write + -ing (final e dropped)",
            "root": "write",
            "type": "Regular spelling change",
            "pos": "Verb (VBG)"
        },
```

```
    "written": {
        "state": "q_irregular",
        "components": "write + irregular past participle form",
        "root": "write",
        "type": "Irregular",
        "pos": "Verb (VBN)"
    }
}

if word not in patterns:
    path.append("q_reject")
    print("Step 3: No known pattern; reject.")
    print("State path:", " -> ".join(path))
    return None

data = patterns[word]
path.append(data["state"])
print("Step 3: Pattern matched ->", data["state"])
print("Step 4: Components =", data["components"])
print("Step 5: Root =", data["root"])
print("Step 6: Classification =", data["type"])
print("Step 7: POS tag =", data["pos"])

path.append("qf")
print("Step 8: Move to final state qf")
print("State path:", " -> ".join(path))
```

```
    print("Normalized form:", data["root"])

    return [
        word, " -> ".join(path), data["components"],
        data["root"], data["type"], data["pos"], data["root"]
    ]


words = ["writes", "writing", "written"]

print("FINITE-STATE MORPHOLOGICAL PARSER")
print("=" * 110)

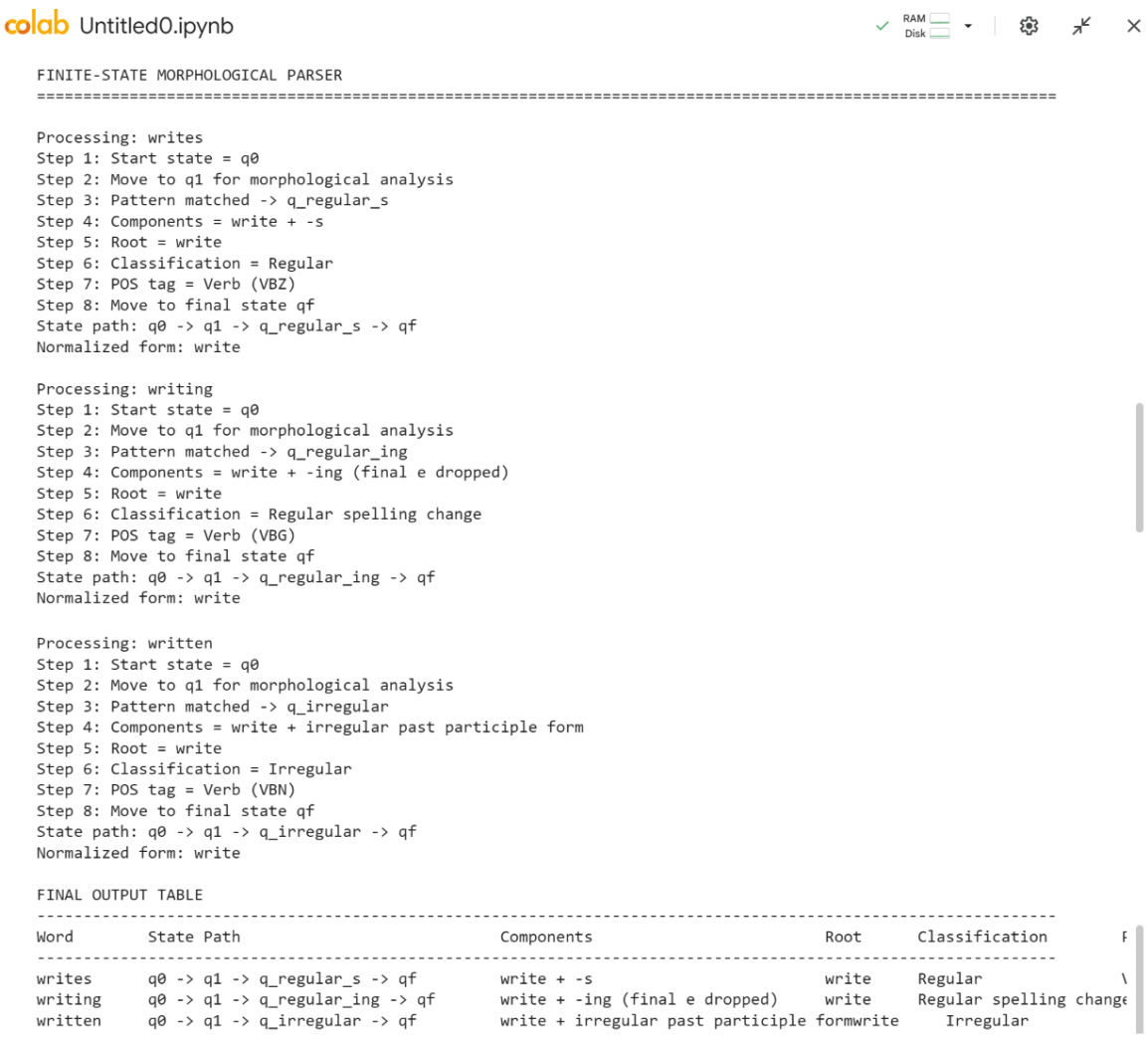
results = []

for word in words:
    result = parse_word(word)
    if result:
        results.append(result)

print("\nFINAL OUTPUT TABLE")
print("-" * 110)
print(f"{'Word':<12}{'State Path':<38}{'Components':<35}"
      f"{'Root':<10}{'Classification':<22}{'POS':<12}")
print("-" * 110)
```

```
for row in results:
    print(f"{row[0]:<12}{row[1]:<38}{row[2]:<35}"
          f"{row[3]:<10}{row[4]:<22}{row[5]:<12}")
```

5.Output



```
colab Untitled0.ipynb

FINITE-STATE MORPHOLOGICAL PARSER
================================================================================================================

Processing: writes
Step 1: Start state = q0
Step 2: Move to q1 for morphological analysis
Step 3: Pattern matched -> q_regular_s
Step 4: Components = write + -s
Step 5: Root = write
Step 6: Classification = Regular
Step 7: POS tag = Verb (VBZ)
Step 8: Move to final state qf
State path: q0 -> q1 -> q_regular_s -> qf
Normalized form: write

Processing: writing
Step 1: Start state = q0
Step 2: Move to q1 for morphological analysis
Step 3: Pattern matched -> q_regular_ing
Step 4: Components = write + -ing (final e dropped)
Step 5: Root = write
Step 6: Classification = Regular spelling change
Step 7: POS tag = Verb (VBG)
Step 8: Move to final state qf
State path: q0 -> q1 -> q_regular_ing -> qf
Normalized form: write

Processing: written
Step 1: Start state = q0
Step 2: Move to q1 for morphological analysis
Step 3: Pattern matched -> q_irregular
Step 4: Components = write + irregular past participle form
Step 5: Root = write
Step 6: Classification = Irregular
Step 7: POS tag = Verb (VBN)
Step 8: Move to final state qf
State path: q0 -> q1 -> q_irregular -> qf
Normalized form: write

FINAL OUTPUT TABLE
----------------------------------------------------------------------------------------------------------------
Word        State Path                          Components                        Root      Classification       F
----------------------------------------------------------------------------------------------------------------
writes      q0 -> q1 -> q_regular_s -> qf       write + -s                        write     Regular              \
writing     q0 -> q1 -> q_regular_ing -> qf     write + -ing (final e dropped)    write     Regular spelling change
written     q0 -> q1 -> q_irregular -> qf       write + irregular past participle formwrite    Irregular
```

6. Result

The finite-state parser recognizes the three specified verb forms, records their state transition paths, differentiates regular and irregular patterns, and normalizes them to write.

7. Interpretation and Justification

- writes: The regular suffix -s marks the third-person singular present-tense form. Its POS tag is VBZ.

- writing: The regular present-participle form is formed by dropping the final e in write before adding -ing. Its POS tag is VBG.
- written: This is an irregular past participle. It must be recognized through an irregular-form rule rather than a simple regular suffix-stripping rule. Its POS tag is VBN.

All three forms map to write, which provides a common representation for text processing and retrieval.

Implementation note: This is a finite-state model for the three specified inputs, using predefined transition conditions. It demonstrates the required transition paths but is not a general English verb parser.

Question 5: Porter Stemmer for Document Retrieval

1. Aim

To implement the Porter Stemming algorithm for relational, relation and relate, apply stemming rules sequentially, display intermediate word forms and obtain a common normalized stem for document retrieval.

2. Theory

The Porter Stemmer reduces words by applying ordered suffix-removal and suffix-replacement rules. It is a rule-based stemming algorithm, not a probabilistic model or a dictionary-based lemmatizer.

The algorithm uses five main steps:

Step 1: Handles plurals and common endings such as -ed and -ing.

Step 2: Replaces certain derivational suffixes, such as -ational and -ation.

Step 3: Removes or replaces additional suffixes.

Step 4: Removes certain suffixes when the stem satisfies the algorithm's measure condition.

Step 5: Handles a final e and certain doubled consonants.

The algorithm uses a measure based on vowel-consonant sequences to decide whether some suffix transformations are permitted.

For the given words, the expected final stem is relat.

3. Algorithm/Procedure

- Install and import the NLTK library.
- Initialize the Porter Stemmer.
- Accept the three specified words.
- Process each word through the ordered Porter steps.
- Record the word before and after every step.
- If a step changes the word, display the intermediate form and identify the applicable rule.
- If a step does not change the word, record that no transformation occurred.
- Obtain the final stem using the Porter Stemmer.
- Display the original word, applied rules, intermediate forms and final stem.

4. Python Code

First, run this installation command in a separate Google Colab cell:

%pip install nltk

Then run the following program in a new cell. It shows all five Porter steps, the intermediate word after each step, and whether the step changed the word.

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

step_methods = [
    ("Step 1a - Plural endings", "_step1a"),
    ("Step 1b - Verb endings", "_step1b"),
    ("Step 1c - Final y", "_step1c"),
    ("Step 2 - Derivational suffixes", "_step2"),
    ("Step 3 - Additional suffixes", "_step3"),
    ("Step 4 - Suffix removal", "_step4"),
    ("Step 5a - Final e", "_step5a"),
    ("Step 5b - Double consonant", "_step5b")
```

```
]

def trace_stemming(word):
    current = word.lower()
    applied_rules = []
    intermediate_forms = [current]

    print("\nOriginal word:", current)
    print("Initial form:", current)

    for step_name, method_name in step_methods:
        previous = current
        method = getattr(stemmer, method_name)
        current = method(current)

        if current != previous:
            decision = "RULE APPLIED"
            detail = previous + " -> " + current
            applied_rules.append(step_name)
            intermediate_forms.append(current)
        else:
            decision = "No change"
            detail = previous + " -> " + current

        print("\n" + step_name)
        print("Before :", previous)
```

```
            print("Decision:", decision)
            print("After  :", current)
            print("Trace  :", detail)

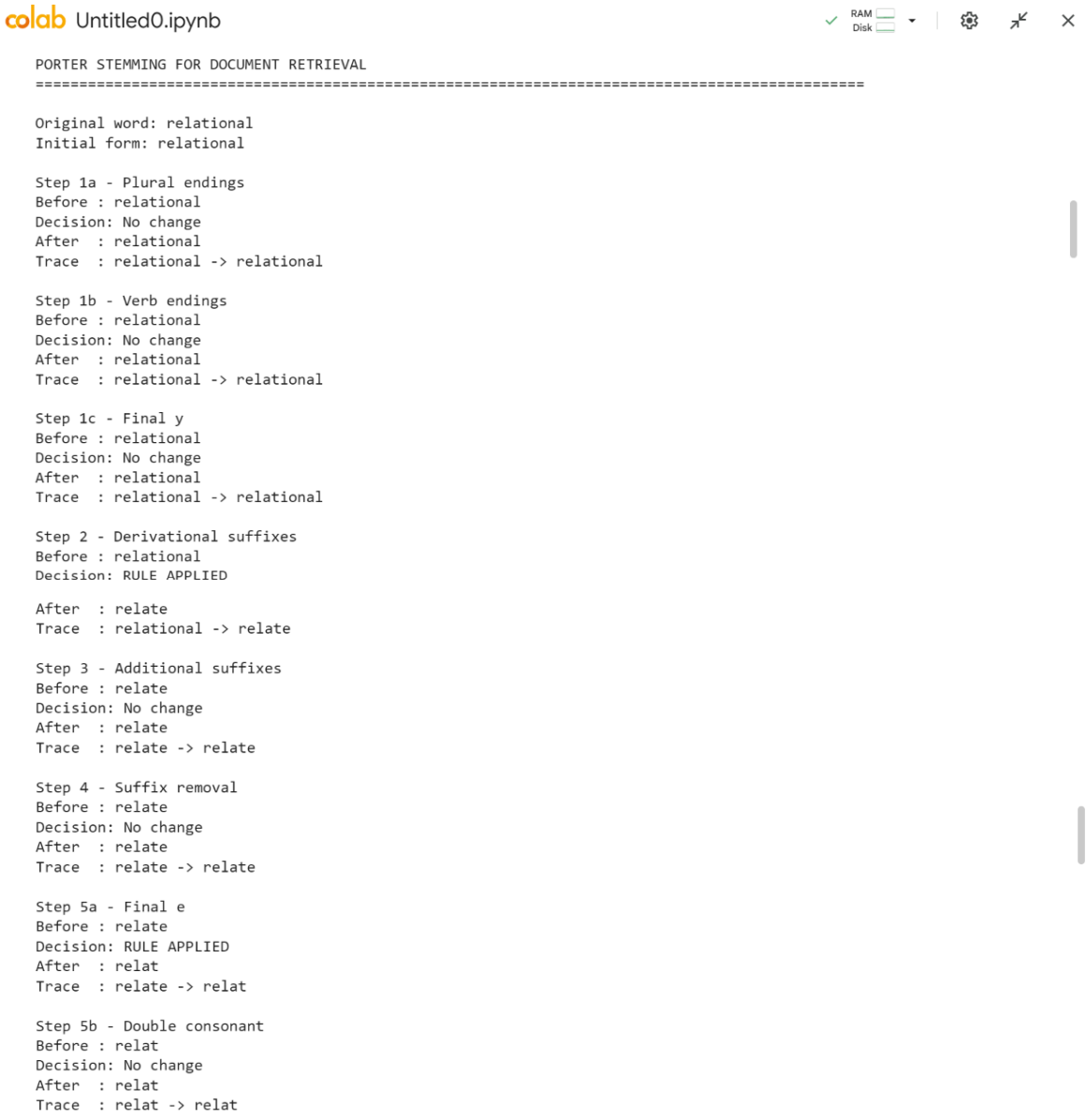
        final_stem = stemmer.stem(word.lower())

        print("\nApplied rules:", applied_rules)
        print("Intermediate forms:", " -> ".join(intermediate_forms))
        print("Final Porter stem:", final_stem)

        return [
            word, '', ''.join(applied_rules),
            " -> ".join(intermediate_forms), final_stem
        ]
words = ["relational", "relation", "relate"]
print("PORTER STEMMING FOR DOCUMENT RETRIEVAL")
print("=" * 95)
results = []
for word in words:
    results.append(trace_stemming(word))
print("\nFINAL OUTPUT TABLE")
print("=" * 95)
print(f"{'Original Word':<18}{'Applied Rules':<40}"
      f"{'Intermediate Forms':<25}{'Final Stem':<12}")
print("-" * 95)
```

```
for row in results:
    print(f"{row[0]:<18}{row[1]:<40}"
          f"{row[2]:<25}{row[3]:<12}")
```

5.Output



```
Applied rules: ['Step 2 - Derivational suffixes', 'Step 5a - Final e']
Intermediate forms: relational -> relate -> relat
Final Porter stem: relat

Original word: relation
Initial form: relation

Step 1a - Plural endings
Before : relation
Decision: No change
After  : relation
Trace  : relation -> relation

Step 1b - Verb endings
Before : relation
Decision: No change
After  : relation
Trace  : relation -> relation

Step 1c - Final y
Before : relation
Decision: No change
After  : relation
Trace  : relation -> relation

Step 2 - Derivational suffixes
Before : relation

Step 2 - Derivational suffixes
Before : relation
Decision: RULE APPLIED
After  : relate
Trace  : relation -> relate

Step 3 - Additional suffixes
Before : relate
Decision: No change
After  : relate
Trace  : relate -> relate

Step 4 - Suffix removal
Before : relate
Decision: No change
After  : relate
Trace  : relate -> relate

Step 5a - Final e
Before : relate
Decision: RULE APPLIED
After  : relat
Trace  : relate -> relat

Step 5b - Double consonant
Before : relat

Decision: No change
After  : relat
Trace  : relat -> relat

Applied rules: ['Step 2 - Derivational suffixes', 'Step 5a - Final e']
Intermediate forms: relation -> relate -> relat
Final Porter stem: relat

Original word: relate
Initial form: relate

Step 1a - Plural endings
Before : relate
Decision: No change
After  : relate
Trace  : relate -> relate

Step 1b - Verb endings
Before : relate
Decision: No change
After  : relate
Trace  : relate -> relate

Step 1c - Final y
Before : relate
Decision: No change
After  : relate
```

```
Trace  : relate -> relate

Step 2 - Derivational suffixes
Before : relate
Decision: No change
After  : relate
Trace  : relate -> relate

Step 3 - Additional suffixes
Before : relate
Decision: No change
After  : relate
Trace  : relate -> relate

Step 4 - Suffix removal
Before : relate
Decision: No change
After  : relate
Trace  : relate -> relate

Step 5a - Final e
Before : relate
Decision: RULE APPLIED
After  : relat
Trace  : relate -> relat

Step 5b - Double consonant
Before : relat
Decision: No change
After  : relat
Trace  : relat -> relat

Applied rules: ['Step 5a - Final e']
Intermediate forms: relate -> relat
Final Porter stem: relat

FINAL OUTPUT TABLE
=============================================================================================
Original Word     Applied Rules                          Intermediate Forms       Final Stem
---------------------------------------------------------------------------------------------
relational        Step 2 - Derivational suffixes, Step 5a - Final erelational -> relate -> relatrelat
relation          Step 2 - Derivational suffixes, Step 5a - Final erelation -> relate -> relatrelat
relate            Step 5a - Final e                      relate -> relat          relat
```

The exact formatting of the final table depends on the console width. The individual trace printed above is the main record of the intermediate transformations.

6. Result

The Porter Stemming program processes all three words through the algorithm's steps and produces the common final stem relat.

7.Justification

- relational: Step 2 replaces the applicable derivational suffix pattern to produce relate. Step 5a removes the final e, producing relat.
- relation: Step 2 transforms the suffix pattern to produce relate. Step 5a removes the final e, producing relat.
- relate: The earlier steps leave the word unchanged. Step 5a removes the final e, producing relat.

The final form relat is a stem produced by the algorithm; it is not necessarily a complete dictionary word. Stemming helps group related terms for retrieval, but it does not guarantee that every word sharing a stem has precisely the same meaning.

Implementation note: The program uses NLTK's Porter Stemmer step methods to display the trace and verifies the final stem with stemmer.stem(). This allows the intermediate output to follow the library's actual rule sequence for these words.